ML Based Automatic Modulation Classification

Department of Electronics and Communication
National Institute of Technology Karnataka
Mangaluru, India

Guide name: Dr. Mandeep Singh

Members:

1. M S S Vinay (231EC229)
2. Merric Simon Lewis (231EC231)
3. Sriram Kiran Akella (231EC262)

April 24, 2026

Table of Contents

	Content	Page number
1	Introduction	5
2	Objectives	6
3	Block Diagram	7
4	Progress and Results 4.1 Methodology 4.2 Simulation Results	11
5	Conclusion and Future Work	18
6	References	19

Abstract

Automatic Modulation Classification (AMC) plays a crucial role in modern wireless communication systems, particularly in applications such as cognitive radio, spectrum monitoring, and electronic warfare. The rapid growth of wireless technologies has increased the need for efficient and reliable techniques to identify modulation schemes from received signals without prior knowledge of transmitter parameters. Traditional signal processing approaches often rely on manually designed features, which may not perform well under noisy and dynamic channel conditions.

This project explores the use of Machine Learning (ML) and Deep Learning (DL) techniques for automatic modulation classification. Conventional ML algorithms such as Support Vector Machines (SVM) and Random Forest are used with extracted signal features, while Deep Learning models like Convolutional Neural Networks (CNN) are employed to automatically learn discriminative features directly from raw signal data. The performance of these models is evaluated using simulated communication signals with different modulation schemes under varying signal-to-noise ratio (SNR) conditions.

Experimental results demonstrate that deep learning-based approaches provide higher classification accuracy and robustness compared to traditional machine learning methods, particularly in low SNR environments. The study highlights the potential of intelligent data-driven techniques for improving the efficiency and reliability of modulation recognition in next-generation wireless communication systems.

CERTIFICATE

I hereby certify that the Minor Project entitled **ML Based Automatic Modulation Classification**, which is being submitted to the National Institute of Technology Karnataka, Surathkal, in partial fulfilment of the requirements for the award of the Degree of B.Tech ECE in Department of Electronics and Communication Engineering is a bonafide report of work carried out by us.

Team Members Name (Roll no.):
MSS Vinay (231EC229)
Merric Simon Lewis (231EC231)
Sriram Kiran Akella (231EC262)

Place: NITK, Surathkal
Date: 24/4/2026

Forwarded by:

Project Supervisor
(Name & Signature)

1. Introduction

Automatic Modulation Classification (AMC) enables a receiver to identify the modulation scheme of a received signal without prior knowledge of transmitter parameters. It serves as an important step between signal detection and demodulation in modern wireless communication systems.

In this project, a Radio Machine Learning dataset is used to perform modulation classification using data-driven techniques. Initially, classical machine learning models are implemented, followed by deep learning-based approaches to improve classification performance, especially under noisy channel conditions.

The RadioML dataset, which was last updated in 2018, consists of more than 2.5 million signal samples covering 24 different modulation schemes. Unlike conventional datasets that contain tabular features, this dataset stores signals as raw I/Q samples. Each signal contains two components: the In-phase (I) and Quadrature (Q) components, with 1024 time samples each.

The modulation schemes present in the dataset include: OOK, 4ASK, 8ASK, BPSK, QPSK, 8PSK, 16PSK, 32PSK, 16APSK, 32APSK, 64APSK, 128APSK, 16QAM, 32QAM, 64QAM, 128QAM, 256QAM, AM-SSB-WC, AM-SSB-SC, AM-DSB-WC, AM-DSB-SC, FM, GMSK, and OQPSK.

Each signal sample is labelled with its corresponding modulation scheme and signal-to-noise ratio (SNR). Using this dataset, modulation classification can be performed using machine learning algorithms such as k-Nearest Neighbors (KNN) and Random Forest classifiers, as well as deep learning models like Convolutional Neural Networks (CNNs).

2. Objectives

The main objectives of this project are to:

- Study and understand Automatic Modulation Classification (AMC) as a key component of modern digital communication receivers.
- Utilize a radio machine learning dataset containing labelled modulation schemes for supervised learning.
- Study the dataset, its features, different SNR values for each modulation scheme
- Randomly select only a few signal samples in order for faster computation and to prevent overfitting. In this case only 32,000 samples were taken to train the model.
- Implement a machine learning–based AMC system using suitable signal representations or extracted features.
- Evaluate the performance of ML classifiers by finding the loss and obtaining the confusion matrix under varying signal-to noise ratio (SNR) conditions.
- Identify the limitations of traditional ML-based AMC, such as dependency on handcrafted features.
- Extend the work toward deep learning–based AMC, enabling automatic feature learning from raw I/Q samples.
- Tuned hyperparameters such as learning rate, number of epochs, and batch size to optimize model performance, and evaluated the results using metrics like accuracy and loss, comparing them with findings reported in research papers.

3. Block Diagram

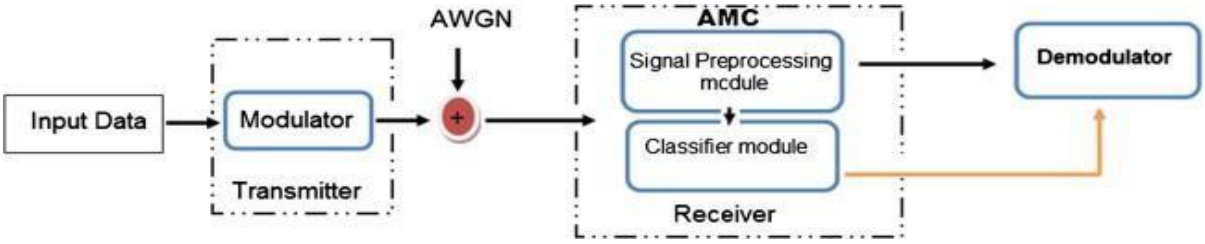


Fig. 1. Block Diagram for Automatic Modulation Classification

Block Diagram Description: -

1. Signal Generation / Dataset

- Modulated signals are generated (BPSK, QPSK, QAM, etc.) •
  In most AMC research, datasets such as RadioML2018 are used.

2. Channel Model

- Signals passing through wireless channel effects:
  - AWGN noise
  - Rayleigh fading
  - Phase offset

3. IQ Sample Extraction

- The receiver captures complex I/Q samples
- These represent the baseband signal.

4. Preprocessing

- Normalization
- Reshaping of IQ data into tensors suitable for neural networks.

5. Feature Extraction (CNN layers)

- CNN extracts patterns from signal structure.

6. Classification Layer

- Fully connected layer predicts the modulation class.

7. Output

- Predicted modulation type

Block Diagram Description (Deep Learning – CNN Based AMC)

Fig. 2. Block Diagram for Deep Learning- CNN

1. Input Representation

- The input to the system consists of complex baseband I/Q samples obtained from modulated signals.
- Each sample is represented as two channels:

    - In-phase (I)
    - Quadrature (Q)
- These are arranged as a 2D tensor, making it suitable for convolutional processing.

2. Initial Convolution Layer

- The first **3×3 convolution layer** processes raw I/Q data.
- It extracts low-level features such as:
    - Signal amplitude variations
    - Phase information
- This is followed by:
    - Batch Normalization (BN) → stabilizes learning
    - ReLU activation → introduces non-linearity

3. Deep Feature Extraction (Convolutional Blocks)

- Multiple convolution layers are stacked to learn hierarchical features.
- Each block typically consists of:
    - Convolution → BN → ReLU
- As depth increases, the network learns:
    - Symbol transitions
    - Constellation patterns
    - Modulation-specific signatures

4. Residual Learning (Skip Connections)

- The architecture uses Residual (ResNet-style) connections.
- Two types of shortcuts are present in your diagram:
    - Identity skip → directly adds input to output
    - Projection skip (1×1 convolution) → matches dimensions when needed

5. Downsampling and Channel Expansion

- Certain convolution layers use:
    - Stride (s=2) for downsampling
    - Increased number of filters (e.g., 64 → 128 → 256)

- This results in:
    - Reduced spatial size
    - Increased feature depth
- Enables the model to capture more complex patterns efficiently

6. Global Feature Aggregation

- A Global Average Pooling layer is applied:
    - Converts feature maps into a compact feature vector
- This reduces:
    - Model complexity
    - Risk of overfitting

7. Fully Connected Layers

- The pooled features are passed through:
    - Dense (Fully Connected) layer
    - Dropout layer for regularization
- These layers learn the final decision boundaries between modulation classes

8. Output Layer

- Final layer uses Softmax activation:
- Produces probability distribution over modulation classes
- The class with highest probability is selected as output

9. Model Outcome

- The CNN automatically learns features directly from raw I/Q data, unlike ML models which rely on manual feature extraction.
- Due to deep feature learning and residual connections:
    - The model achieves higher classification accuracy
    - Performs better under noisy channel conditions

4. Progress and Results

4.1 Methodology

- A number of 60000 signal samples were taken to train the model. We have taken only 8 modulation schemes to be classified which are 4ASK, BPSK, QPSK, 16PSK, 16QAM, FM, AM-DSB-WC, 32APSK for easy visualization and understanding.
- Each sample had an SNR between -20 to 20, a desired SNR was chosen within this range (-10). Then X_data list was made with these 8 modulation schemes and -10 desired SNR value. The Y_data was also taken in this similar way.
- The modulation labels where one hot encoded (binary) in the dataset, the program later gave an integer value for training.
- The data was given an 80:20 split, 80% was used to train the model and the rest 20% was used to test the model.
- Models like KNNs and Random Forest classifier (with and without hyperparameter tuning were used).
- Later a deep learning model, was made to classify the signal for a particular SNR value.

4.2 Simulation results

1. Dataset Loading

The project uses the RadioML2018 dataset, which is a widely used benchmark dataset for automatic modulation classification. The dataset is stored in HDF5 format and contains three main components:

Variable	Description
X	IQ signal samples
Y	One-hot encoded modulation labels
Z	Signal-to-noise ratio (SNR) values

```
Successfully loaded 600000 samples.
Shape of X (features): (600000, 1024, 2)
Shape of Y (labels): (600000, 24)
Shape of Z (SNR values): (600000, 1)
```

Listing 1. Successfully loaded 600000 samples along with shape of dataset

2. Defining Modulation Classes

The RadioML dataset contains 24 different modulation schemes out of which only 8 modulation schemes were chosen to be classified which are 4ASK, BPSK, QPSK, 16PSK, 16QAM, FM, AM-DSB-WC, 32APSK.

3. Selecting SNR and Classes

The dataset contains signals at multiple SNR levels, ranging from:
+20 dB (very clean signal) to **−20 dB (very noisy signal)** For this experiment:

- Only **SNR = −10 dB** is selected
- This helps evaluate the classifier under noisy conditions

4. Reshaping Signal Data and Converting Labels to DataFrame

The IQ samples are reshaped into a 2D format suitable for machine learning models.
The labels are stored as one-hot encoded vectors which is a binary vector where only one element is "hot" (set to 1) and all others are "cold" (set to 0).

	4ASK	BPSK	QPSK	16PSK	16QAM	FM	AM-DSB-WC	32APSK
0	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0
4	1	0	0	0	0	0	0	0
...	...	...	...	...	...	..	...	...
32763	0	0	0	0	1	0	0	0
32764	0	0	0	0	1	0	0	0
32765	0	0	0	0	1	0	0	0
32766	0	0	0	0	1	0	0	0
32767	0	0	0	0	1	0	0	0

[32768 rows x 8 columns]

Listing 2. Converts labels into a pandas DataFrame

5. Converting One-Hot Labels to Class Labels

Machine learning models require single integer labels instead of one-hot vectors. Implemented using the numpy.argmax() function, which returns the index of the first occurrence of the maximum value in an array or list.

```
Modulation 1: 4096 samples
Modulation 3: 4096 samples
Modulation 4: 4096 samples
Modulation 6: 4096 samples
Modulation 12: 4096 samples
Modulation 21: 4096 samples
Modulation 19: 4096 samples
Modulation 9: 4096 samples
```

Listing 3. Distribution of modulation samples

6. Flattening Data for ML Models

Traditional ML models such as KNN and Random Forest require 1D feature vectors. Therefore, the 3D signal data has original dimensions as (32767 x 1024 x 2) is converted to dimensions (32767 x 2048) using flatten function.

7. K-Nearest Neighbors

The K-Nearest Neighbors (KNN) algorithm classifies a signal based on the majority class among its nearest neighbors.

Steps:

1. Compute Euclidean distance between signals
2. Select nearest neighbors
3. Assign most frequent modulation class

The trained KNN model predicts modulation classes for the test dataset, and accuracy measures the proportion of correct predictions.

```
Test accuracy 0.5636252670125115
```

Listing 4. Model accuracy for k = 5

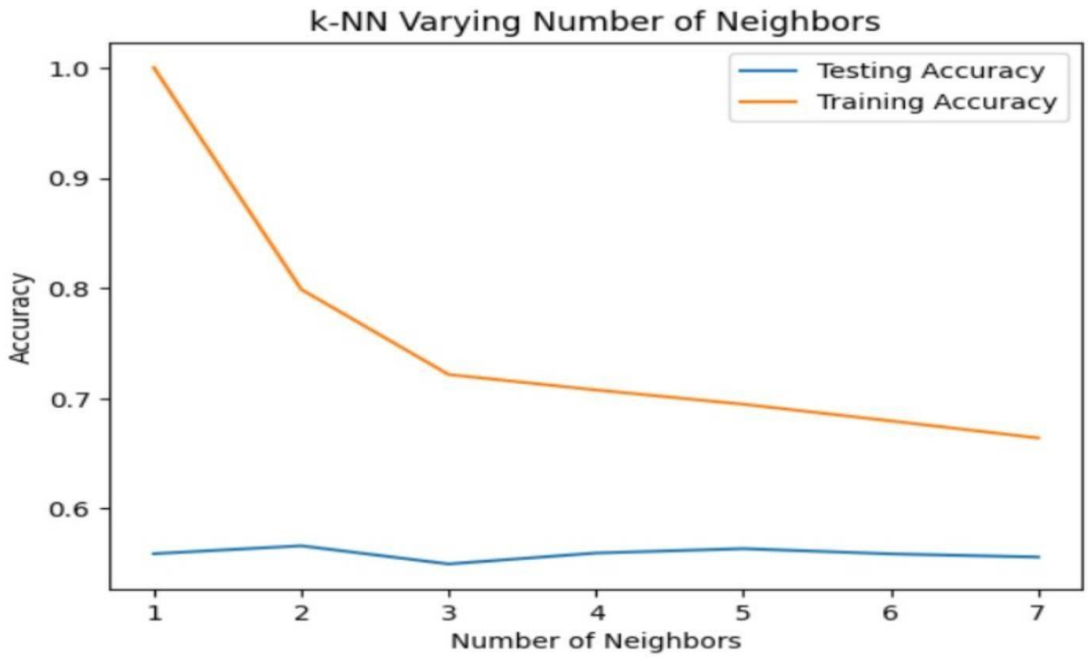


Fig. 3. Effect of the number of neighbors (k) on the training and testing accuracy of KNN classifier

8.Random Forest Model and Hyper parameter tuning

Random Forest is an ensemble learning method that uses multiple decision trees. The model learns relationships between signal features and modulation classes.

Parameter	**Description**
n_estimators	number of trees
max_depth	tree depth
n_jobs=-1	use all CPU cores

The final prediction is obtained by majority voting of all trees.
The trained model predicts modulation types for test samples.

```
Random Forest Accuracy: 0.6359475129691792
```

Listing 5. Random Forest model accuracy

Therefore, typically **Random Forest performs better than KNN** due to ensemble learning.

Different combinations of parameters are tested to find the best model configuration.

- From the results shown in the table, it indicates that using a larger number of trees (**n_estimators = 200)** allows the model to better capture complex patterns in the signal data, while allowing the trees to grow deeper **(max_depth = None**) enables more detailed decision boundaries. The parameter **min_samples_leaf = 4** helps prevent overfitting by ensuring that each leaf node contains a sufficient number of samples.

	n_estimators	max_depth	min_samples_leaf	accuracy
20	200	NaN	4	0.660055
25	200	20.0	2	0.650137
18	200	NaN	1	0.649527
26	200	20.0	4	0.649222
19	200	NaN	2	0.648459
24	200	20.0	1	0.644492
11	100	NaN	4	0.639915
9	100	NaN	1	0.635948
10	100	NaN	2	0.635337
17	100	20.0	4	0.631828
16	100	20.0	2	0.628166
15	100	20.0	1	0.618554
0	50	NaN	1	0.615349
2	50	NaN	4	0.614281
1	50	NaN	2	0.612603
7	50	20.0	2	0.610009
8	50	20.0	4	0.606042
6	50	20.0	1	0.598261
21	200	10.0	1	0.440189
22	200	10.0	2	0.432713
23	200	10.0	4	0.429814
12	100	10.0	1	0.429509
14	100	10.0	4	0.425999
13	100	10.0	2	0.424474
4	50	10.0	2	0.417455
3	50	10.0	1	0.414403
5	50	10.0	4	0.403723

Listing 6. 66% Accuracy after Hyperparameter tuning

9. Deep Learning- CNN Model Results

A signet model was used to classify the signal at SNR= -10db.

- Parameters Analysis- The CNN model has 2.81 million parameters (~10.73 MB), of which almost all are trainable, enabling it to learn complex modulation features effectively. The small number of non-trainable parameters comes from Batch Normalization, which helps stabilize training without adding to learning complexity.

```
Total params: 2,812,360 (10.73 MB)
Trainable params: 2,807,880 (10.71 MB)
Non-trainable params: 4,480 (17.50 KB)
```

Listing 7. Parameters Trained

- Initially, the CNN model achieved **~85% accuracy with 10 epochs**, indicating that it was able to learn basic modulation features. By increasing the training to **30 epochs**, the model had more iterations to refine its weights and capture deeper signal patterns, resulting in a significant improvement to **~98% accuracy**.

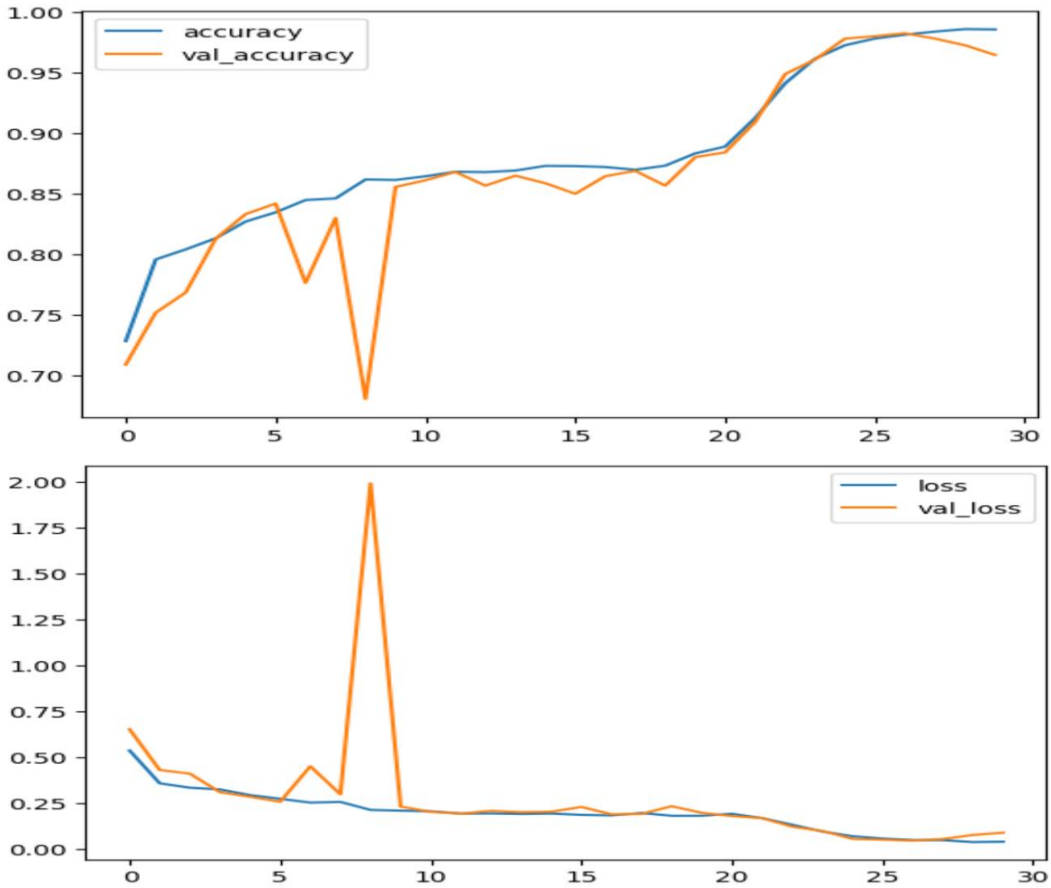
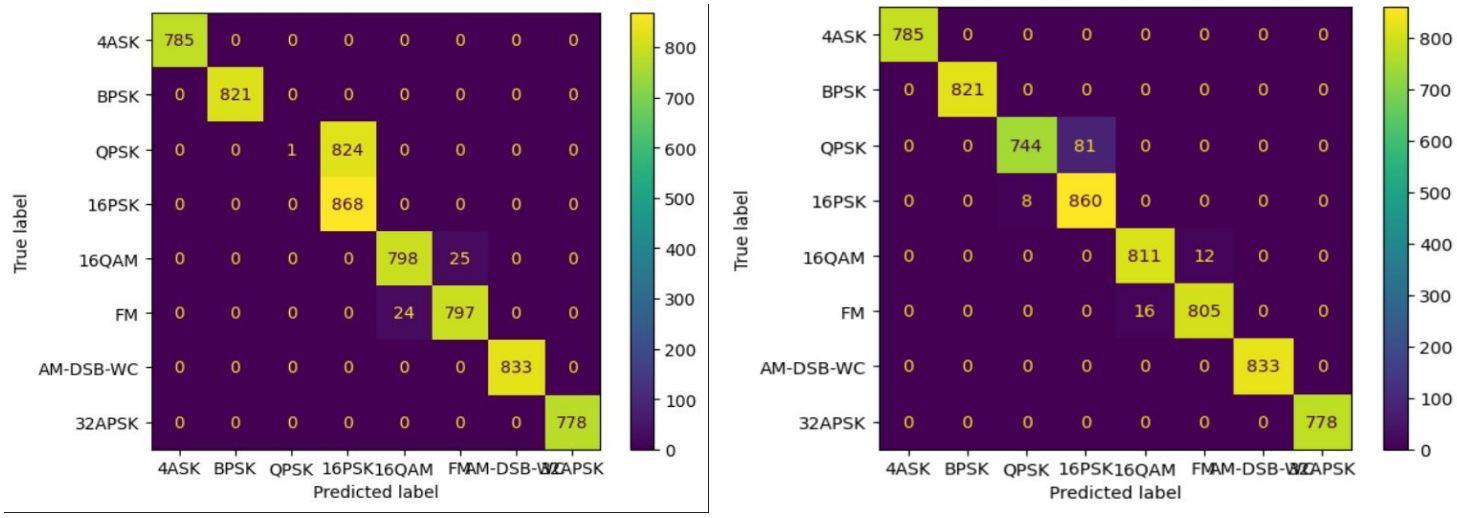


Fig. 4. CNN Model Training and Validation Accuracy and Loss vs. Epochs

10. Confusion Matrix for the Deep learning model

In the confusion matrix, **rows represent the actual modulation classes**, while **columns represent the predicted modulation classes**.

- **Diagonal elements** represent correctly classified samples.
- **Off-diagonal elements** represent misclassified samples.



Listing 8. Confusion Matrix for 85% and 98% accuracy

- The improvement in accuracy from **~85% to ~98%** is clearly reflected in the confusion matrices. In the 85% case, there are noticeable misclassifications, particularly between similar modulation schemes such as QPSK and 16PSK, and 16QAM and FM, indicating that the model had not fully learned their distinguishing features.

- After training for 30 epochs, the confusion matrix becomes almost perfectly diagonal, with significantly reduced off-diagonal values. This shows that the model has learned more discriminative features, leading to **near-perfect classification and minimal confusion between modulation types**, thereby achieving higher accuracy.

5. Conclusion and Future work

- The system initially achieved ~66% accuracy using ML models (KNN, Random Forest). With the implementation of a CNN-based architecture on raw I/Q data and training up to 30 epochs, the accuracy significantly improved to **~98%**, with reduced misclassification as seen in the confusion matrix.
- The CNN model effectively learns complex signal features and minimizes confusion between similar modulation schemes, resulting in more robust and reliable classification performance.
- **Future Work:** Extend the model to classify signals across different SNR levels by incorporating SNR-based subclassification, improving performance under noisy real-world conditions.

6. References

[1] M. Usman and J.-A. Lee, "AMC-IoT: Automatic modulation classification using efficient convolutional neural networks for low powered IoT devices," in Proceedings of the IEEE International Conference on Information and Communication Technology Convergence (ICTC), 2020, pp. 288–293.

[2] X. Y. Zeng, A. B. Clark, and M. Q. Smith, "Deep learning-based modulation classification using CNNs," in Proc. IEEE Global Communications Conference, Singapore, 2023, pp. 123–12.